Week 11 Asynchronous Materials Guide - Dictionaries

This Week's Focus

Building on lecture: The lecture introduced basic dictionary operations. This guide expands into nested dictionaries, advanced patterns, and common use cases.

- Review: Creating, accessing, and modifying dictionaries
- New: Nested dictionaries for complex data
- New: Dictionary comprehensions
- New: Common patterns (counting, grouping, lookups)
- New: Combining dictionaries with lists

Start Here - 10 Minutes

Video: Python Dictionaries

Corey Schafer - Dictionaries

- Quick reminder of the in person lecture
- Watch through video, paying attention to the examples
- Understand key:value data

Core References

W3Schools - Python Dictionaries

https://www.w3schools.com/python/python_dictionaries.asp

- Quick reference for syntax
- Interactive examples

Real Python - Dictionaries

https://realpython.com/python-dicts/

- Comprehensive guide
- Read up to but not Creating Custom Dictionary-Like Classes
- Pay attention to the Built in functions and Iterations

Review: Dictionary Basics

This was covered in lecture. Quick review before moving to new material.

Creating and Accessing

```
# Create dictionary
student = {
    "name": "Alice",
    "age": 20,
    "major": "CS"
}

# Access values
print(student["name"])  # "Alice"
print(student.get("email", "No email"))  # Safe access

# Add or update
student["gpa"] = 3.8
student["age"] = 21

# Check if key exists
if "name" in student:
    print("Name exists")

# Get length
print(len(student))  # Number of key-value pairs
```

Looping Through Dictionaries

```
grades = {"Alice": 92, "Bob": 85, "Charlie": 78}
```

```python
# Loop through keys
for name in grades:
    print(name)

# Loop through values
for grade in grades.values():
    print(grade)

# Loop through key-value pairs (most common)
for name, grade in grades.items():
    print(f"{name}: {grade}")
```

Nested Dictionaries

Dictionaries can contain other dictionaries! This is powerful for organizing complex data.

W3Schools - Nested Dictionaries

https://www.w3schools.com/python/python_dictionaries_nested.asp

- Simple examples of nested structures
- How to access nested values

Basic Nested Dictionary

```python
# Each student is a dictionary
students = {
    "Alice": {
        "age": 20,
        "major": "CS",
        "gpa": 3.8
    },
    "Bob": {
        "age": 19,
        "major": "Math",
        "gpa": 3.6
    }
}

# Access nested data
```

```
print(students["Alice"])  # Whole dictionary
print(students["Alice"]["major"])  # "CS"

# Modify nested data
students["Alice"]["gpa"] = 3.9

# Add new nested dictionary
students["Charlie"] = {
    "age": 21,
    "major": "Physics",
    "gpa": 3.7
}
```

Looping Through Nested Dictionaries

```
students = {
    "Alice": {"age": 20, "major": "CS", "gpa": 3.8},
    "Bob": {"age": 19, "major": "Math", "gpa": 3.6}
}

# Loop through nested structure
for name, info in students.items():
    print(f"\n{name}:")
    print(f"  Age: {info['age']}")
    print(f"  Major: {info['major']}")
    print(f"  GPA: {info['gpa']}")

# Calculate average GPA
total_gpa = 0
for info in students.values():
    total_gpa += info["gpa"]
average = total_gpa / len(students)
print(f"\nAverage GPA: {average:.2f}")
```

Practical Example: Restaurant Menu

```
menu = {
    "burger": {
        "price": 12.99,
        "calories": 850,
        "category": "main"
    },
```

```
    "fries": {
        "price": 4.99,
        "calories": 400,
        "category": "side"
    },
    "soda": {
        "price": 2.99,
        "calories": 150,
        "category": "drink"
    }
}

# Display menu
print("MENU")
print("=" * 40)
for item, details in menu.items():
    print(f"\n{item.title()}")
    print(f"  Price: ${details['price']}")
    print(f"  Calories: {details['calories']}")

# Find all main dishes
print("\nMain Dishes:")
for item, details in menu.items():
    if details["category"] == "main":
        print(f"  {item}: ${details['price']}")
```

Common Patterns with Dictionaries

Pattern 1: Counting Occurrences

```
# Count how many times each item appears
votes = ["Alice", "Bob", "Alice", "Charlie", "Bob", "Alice"]

vote_count = {}
for name in votes:
    if name in vote_count:
        vote_count[name] += 1
    else:
        vote_count[name] = 1

print(f"Vote counts: {vote_count}")
```

```
# Find winner
max_votes = 0
winner = ""
for name, count in vote_count.items():
    if count > max_votes:
        max_votes = count
        winner = name

print(f"Winner: {winner} with {max_votes} votes")
```

Pattern 2: Grouping Data

```
# Group students by major
students = [
    {"name": "Alice", "major": "CS"},
    {"name": "Bob", "major": "Math"},
    {"name": "Charlie", "major": "CS"},
    {"name": "Diana", "major": "Physics"},
    {"name": "Eve", "major": "Math"}
]

# Build dictionary of lists
by_major = {}
for student in students:
    major = student["major"]
    name = student["name"]

    if major not in by_major:
        by_major[major] = []

    by_major[major].append(name)

# Display grouped results
for major, names in by_major.items():
    print(f"{major}: {names}")
```

Pattern 3: Lookup Tables

```
# Quick lookups with dictionaries
phone_book = {
    "Alice": "555-1234",
    "Bob": "555-5678",
    "Charlie": "555-9012"
}
```

```
# Fast lookup (much faster than searching a list!)
name = "Bob"
if name in phone_book:
    print(f"{name}'s phone: {phone_book[name]}")
else:
    print(f"{name} not found")

# Reverse lookup (slower, but possible)
target_phone = "555-5678"
for name, phone in phone_book.items():
    if phone == target_phone:
        print(f"{target_phone} belongs to {name}")
        break
```

Pattern 4: Aggregating Data

```
# Calculate statistics by category
sales = [
    {"item": "laptop", "category": "electronics", "price": 999},
    {"item": "mouse", "category": "electronics", "price": 25},
    {"item": "desk", "category": "furniture", "price": 299},
    {"item": "chair", "category": "furniture", "price": 199}
]

# Sum prices by category
category_totals = {}
for sale in sales:
    category = sale["category"]
    price = sale["price"]

    if category not in category_totals:
        category_totals[category] = 0

    category_totals[category] += price

# Display results
for category, total in category_totals.items():
    print(f"{category}: ${total}")
```

Combining Dictionaries and Lists

List of Dictionaries

```
# Common pattern: list of records
inventory = [
    {"name": "Laptop", "price": 999, "quantity": 5},
    {"name": "Mouse", "price": 25, "quantity": 50},
    {"name": "Keyboard", "price": 75, "quantity": 30}
]

# Display inventory
print("INVENTORY")
print("-" * 40)
for item in inventory:
    print(f"{item['name']}: ${item['price']} ({item['quantity']} in stock)")

# Calculate total inventory value
total_value = 0
for item in inventory:
    item_value = item["price"] * item["quantity"]
    total_value += item_value

print(f"\nTotal inventory value: ${total_value}")
```

Dictionary of Lists

```
# Store multiple values per key
grade_book = {
    "Alice": [85, 92, 88, 90],
    "Bob": [78, 85, 80, 82],
    "Charlie": [92, 95, 89, 94]
}

# Calculate averages
print("Student Averages:")
for student, grades in grade_book.items():
    avg = sum(grades) / len(grades)
    print(f"{student}: {avg:.1f}")

# Find highest single grade
```

```
highest = 0
top_student = ""
for student, grades in grade_book.items():
    max_grade = max(grades)
    if max_grade > highest:
        highest = max_grade
        top_student = student

print(f"\nHighest grade: {top_student} with {highest}")
```

Dictionary Methods Reference

Essential Dictionary Methods

```
# Create
my_dict = {"key": "value"}

# Access
value = my_dict["key"]              # Raises KeyError if missing
value = my_dict.get("key")          # Returns None if missing
value = my_dict.get("key", "default")  # Returns default if missing

# Add/Update
my_dict["new_key"] = "new_value"

# Remove
value = my_dict.pop("key")          # Remove and return value
my_dict.clear()                     # Remove all items

# Check
if "key" in my_dict:                # Check if key exists
    print("Found")

# Iterate
for key in my_dict:                 # Loop through keys
for value in my_dict.values():      # Loop through values
for key, value in my_dict.items():  # Loop through pairs

# Information
len(my_dict)                        # Number of key-value pairs
```

```
list(my_dict.keys())                # Get all keys as list
list(my_dict.values())              # Get all values as list
list(my_dict.items())               # Get all pairs as list
```

Common Mistakes and Solutions

Mistake 1: KeyError When Accessing

```
# WRONG - crashes if key doesn't exist
student = {"name": "Alice", "age": 20}
print(student["email"])  # KeyError!

# CORRECT - use get() for safe access
email = student.get("email", "No email")
print(email)

# CORRECT - check first
if "email" in student:
    print(student["email"])
else:
    print("No email")
```

Mistake 2: Modifying Dictionary While Looping

```
# WRONG - causes problems
inventory = {"apple": 5, "banana": 0, "orange": 3}
for item, qty in inventory.items():
    if qty == 0:
        inventory.pop(item)  # ERROR!

# CORRECT - loop through copy of keys
inventory = {"apple": 5, "banana": 0, "orange": 3}
for item in list(inventory.keys()):
    if inventory[item] == 0:
        inventory.pop(item)
```

```
print(inventory)  # {"apple": 5, "orange": 3}
```

Mistake 3: Forgetting Dictionary Keys Are Case-Sensitive

```
# These are DIFFERENT keys!
student = {
    "Name": "Alice",
    "name": "Bob"
}

print(student["Name"])  # "Alice"
print(student["name"])  # "Bob"

# Be consistent with your key names!
```

When to Use Dictionaries

Dictionaries vs Lists

Use Dictionaries When:

- You need to look things up by name/key
- Data has named attributes
- You need fast lookups
- Relationships between data matter more than order

Use Lists When:

- Order matters
- Simple sequence of similar items
- You need to access by position
- Items are all the same type of thing

Comparison Example

```python
# List - good for ordered data
test_scores = [85, 92, 78, 95, 88]
print(f"First test: {test_scores[0]}")
print(f"Average: {sum(test_scores) / len(test_scores):.1f}")

# Dictionary - good for labeled data
student = {
    "name": "Alice",
    "id": 12345,
    "major": "CS",
    "gpa": 3.8
}
print(f"Student: {student['name']}")
print(f"Major: {student['major']}")

# List of dictionaries - structured records
students = [
    {"name": "Alice", "gpa": 3.8},
    {"name": "Bob", "gpa": 3.6},
    {"name": "Charlie", "gpa": 3.9}
]
for student in students:
    print(f"{student['name']}: {student['gpa']}")
```